

操作系统

第二章 进程管理

第二部分 (6) 盘交换区

同济大学计算机系

一、基本概念 和 基础的进程图像换入换出操作

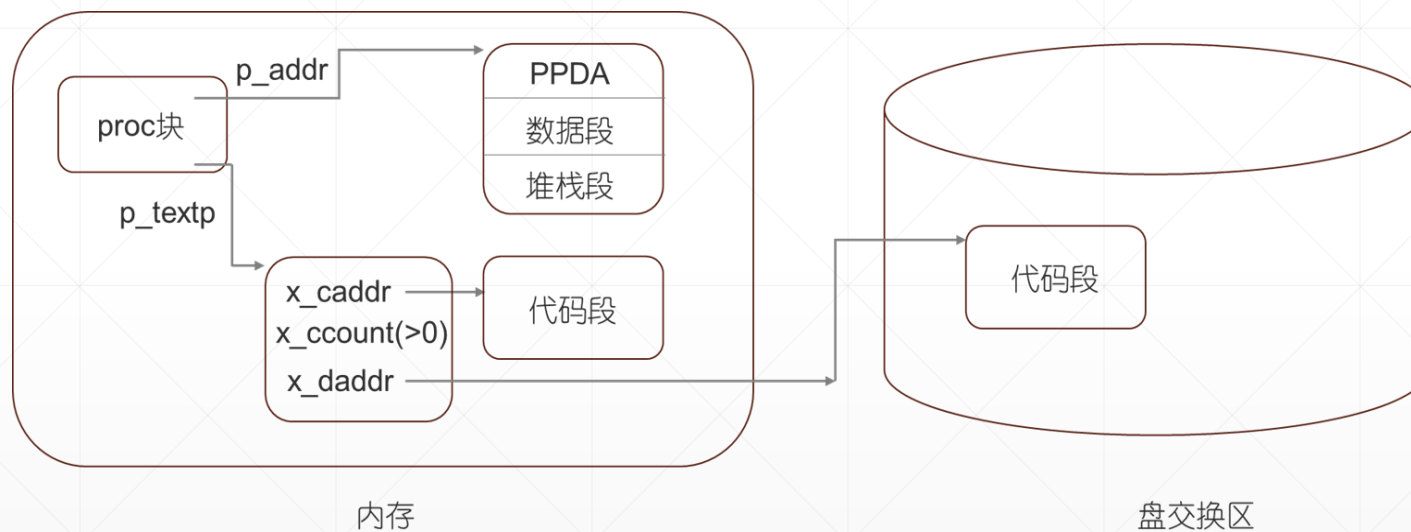
- **可执行存储器** 是用来存放进程图像的存储单元, 包括 内存 和 盘交换区 [注1].
 - 内存中的进程图像可执行, 因为每个单元都有物理地址。
 - 盘交换区中的图像不可执行。运行前, 必需为其分配内存。
- 盘交换区, swap area, 是主磁盘上的一块连续区域, 容纳内存装不下的进程图像 [注2]。
- 换入操作, swap in: 把盘交换区上的进程放进内存。
- 换出操作, swap out: 把内存中的进程放进盘交换区。
- 换出原因: 内存太挤了。换入原因: 内存有空啦。

注1: 与之相对的存储器, 放文件。

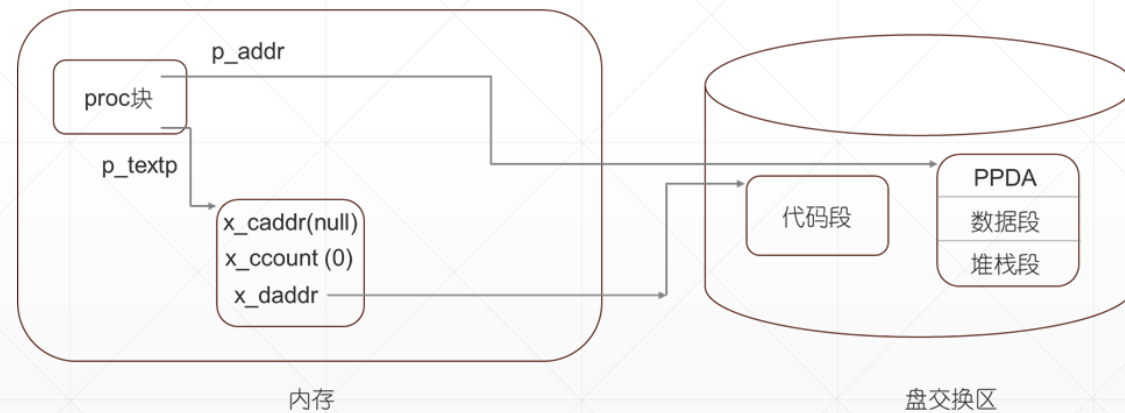
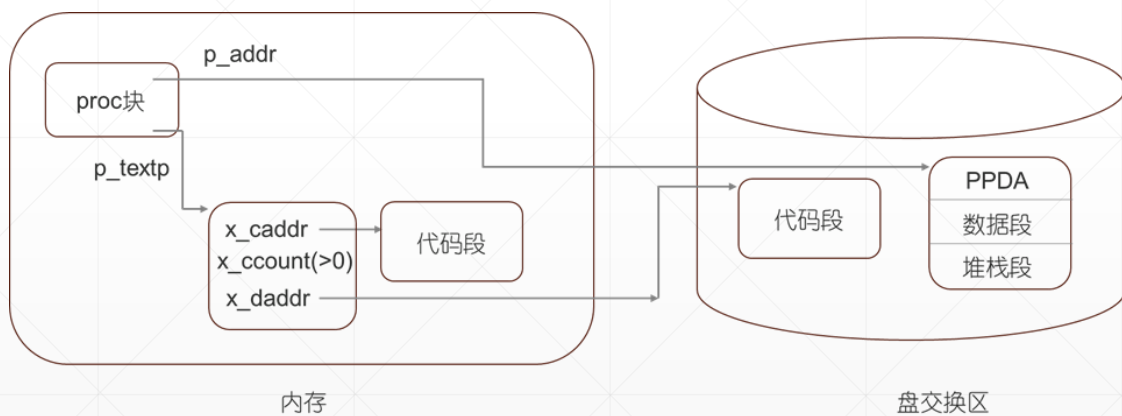
注2: 一些系统, 用文件 (swap file) 装换出进程图像。

Unix V6++ 盘交换区: [18000,20000)扇区

内存中的进程, SLOAD == 1



盘交换区上的进程, $SLOAD == 0$



换入操作 Swap In (0#进程换入进程pSelected)

1、计算进程图像需要的内存空间

```
size = pSelected->p_size;    // == 可交换部分尺寸  
if ( pSelected->p_textp != NULL && 0 == pSelected->p_textp->x_ccount )    // 没有为代码段分配内存  
    size += pSelected->p_textp->x_size;    // += 代码段长度
```

2、为进程图像分配内存

```
desAddress = Kernel::Instance().GetUserPageManager().AllocMemory(size);
```

3、读入 (Buf::B_READ) 正文段

```
if(pSelected->p_textp != NULL && 0 == pSelected->p_textp->x_ccount )  
    bufMgr.Swap(pText->x_daddr, desAddress, pText->x_size, Buf::B_READ)
```

4、读入盘交换区上的可交换部分

```
desAddress += pText->x_size;  
bufMgr.Swap(pSelected->p_addr, desAddress, pSelected->p_size, Buf::B_READ)
```

5、释放可交换部分占用的盘交换区空间

```
Kernel::Instance().GetSwapperManager().FreeSwap(pSelected->p_size, pSelected->p_addr);
```

6、更新PCB

```
pSelected->p_addr = desAddress;                pSelected->p_time = 0;  
pSelected->p_flag |= Process::SLOAD;           修改 x_ccount 和 x_caddr;
```

换出操作 Swap Out

0#进程换出进程pProcess的图像，为盘交换区上的进程图像腾空间：

XSwap(pProcess, bFreeMemory==true, size==0)

- 将进程pProcess的可交换部分换出到盘交换区上，释放内存复本
- size==0, IO整个可交换部分（调用Swap函数） 更新PCB

现运行进程扩展图像（堆栈扩展和malloc），内存不够：

XSwap(pProcess, true, size==oldSize)

- 将现运行进程pProcess的可交换部分换出到盘交换区上，释放内存复本
- IO进程图像扩展前的可交换部分（调用Swap函数） 更新PCB

父进程创建子进程（fork），内存不够：

XSwap(pProcess, false, 0)

- 父进程将可交换部分复制到盘交换区上，内存复本不释放（内存中父进程可交换部分，盘交换区子进程可交换部分）
- size==0, IO整个可交换部分（调用Swap函数） 更新PCB

Swap, 进程图像 IO

`bool BufferManager::Swap(int blkno, unsigned long addr, int count, enum Buf::BufFlag flag)`

入口参数:

`blkno`: 盘交换区上的起始扇区号;

`addr`: 内存起始地址

`count`: 进程图像的长度, 单位是字节;

`flag`: `Buf::B_READ(0x02)`, 磁盘上的进程图像复制进内存;
`Buf::B_WRITE(0x01)`, 反过来。

返回值:

`true`, IO成功; `false`, IO失败;

FIFS的硬盘
IO请求队列



IO请求队列队首是正在执行的IO操作。完成后, 硬盘发中断请求, 系统执行中断处理程序, 中断处理程序:

- `WakeUpAll(&队首Buf)`, 唤醒等待IO结束的进程。
- 摘除队首已完成的IO请求, 启动队列中的下一个IO操作。

- 1、构造IO请求块 `Swbuf`, 打包入口参数, 送IO请求队列尾
- 2、入睡, 同步等待IO操作结束。
`u.u_procp->Sleep (&Swbuf, ProcessManager::PSWP)`
- 3、返回

待所有先前IO操作完成, 中断处理程序向硬盘发IO命令
... 硬盘控制器执行进程图像复制操作 ...
IO完成后, 硬盘发中断请求, 系统执行中断处理程序,
`WakeUpAll(&Swbuf)` 唤醒我们的进程



```
bool BufferManager::Swap(int blkno, unsigned long addr, int count, enum Buf::BufFlag flag)
{
```

```
    User& u = Kernel::Instance().GetUser();
```

```
    X86Assembly::CLI();
```

SwBuf是一个缓存控制块，用来执行进程图像换入换出操作。
SwBuf中登记着换入、换出操作的IO参数。

```
    while ( this->SwBuf.b_flags & Buf::B_BUSY )    // SwBuf 锁住了，其它进程正在使用
```

```
    {
```

```
        this->SwBuf.b_flags |= Buf::B_WANTED; // 置等待使用标识
```

```
        u.u_procp->Sleep( (unsigned long)&SwBuf, ProcessManager::PSWP ); // 入睡等待
```

```
    }
```

```
    this->SwBuf.b_flags = Buf::B_BUSY | flag; // 上锁（置忙标识），表示SwBuf正在使用。
```

```
    this->SwBuf.b_dev = DeviceManager::ROOTDEV;    // 访问主硬盘
```

```
    this->SwBuf.b_wcount = count; // 进程图像的尺寸
```

```
    this->SwBuf.b_blkno = blkno; // 磁盘起始扇区号
```

```
    this->SwBuf.b_addr = (unsigned char *)addr; // 内存首地址
```

置读写标识，换入操作是读，换出操作是写

```
enum BufFlag
{
    B_WRITE = 0x1,
    B_READ  = 0x2,
    B_DONE  = 0x4,
    B_ERROR = 0x8,
    B_BUSY  = 0x10,
    B_WANTED = 0x20,
    B_ASYNC = 0x40,
    B_DELWRI = 0x80
};
```

1、获得SwBuf的使用权

2、设置IO参数

3、发IO命令（将SwBuf送入IO请求队列）

```
    this->m_DeviceManager->GetBlockDevice(Utility::GetMajor(this->SwBuf.b_dev)).Strategy(&this->SwBuf);
```




4、入睡、等待IO操作结束

```
X86Assembly::CLI();
while ( (this->SwBuf.b_flags & Buf::B_DONE) == 0 )
{
    u.u_procp->Sleep((unsigned long)&SwBuf, ProcessManager::PSWP);
}
```

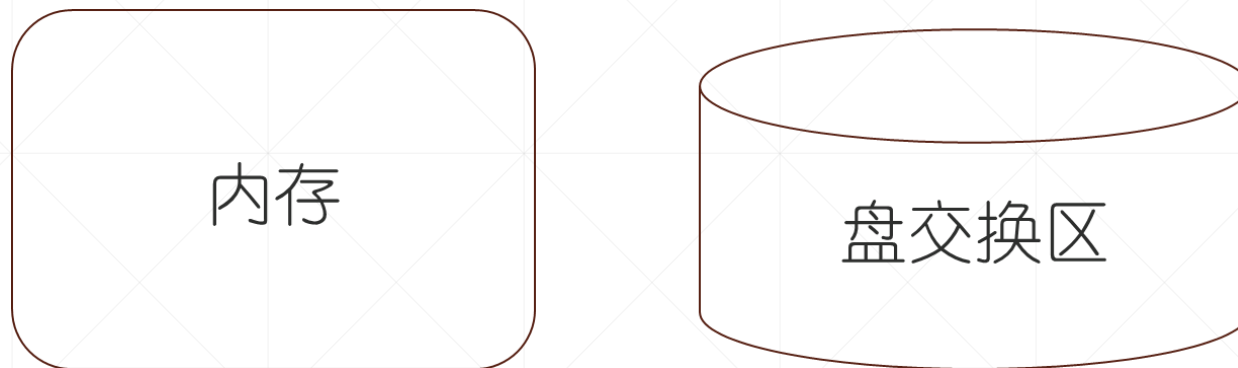
IO（换入/换出操作）完成后，磁盘中断处理程序
(1) 置1 b_flags中的 B_DONE标识 (2) WakeUpAll(&SwBuf), 唤醒执行Swap函数，等待IO完成的进程

5、释放SwBuf的使用权

```
if ( this->SwBuf.b_flags & Buf::B_WANTED )
{
    Kernel::Instance().GetProcessManager().WakeUpAll((unsigned long)&SwBuf);
} // Swbuf使用完毕，唤醒等待使用的其它进程。其中，最先运行的进程将获得Swbuf的使用权
X86Assembly::STI();

this->SwBuf.b_flags &= ~(Buf::B_BUSY | Buf::B_WANTED); // 解锁，清等待使用标识
if ( this->SwBuf.b_flags & Buf::B_ERROR )
{
    return false;
}
return true;
}
```

二、系统实现



可执行程序管理策略：

- (1) 内存可以容纳全部进程图像时，盘交换区不用。
 - (2) 内存不足以容纳全部进程图像时，最优布局：**SWAIT**放盘交换区，腾空间给**SRUN**和**SSLEEP**。
 - (3) 若内存不足以容纳全部**SRUN**和**SSLEEP**，这些进程轮流使用内存资源。
- 中级调度规则，**p_time FIFO**。优先换入盘交换区上驻留最久的**SRUN**进程；必要时，换出最先进入内存的进程。

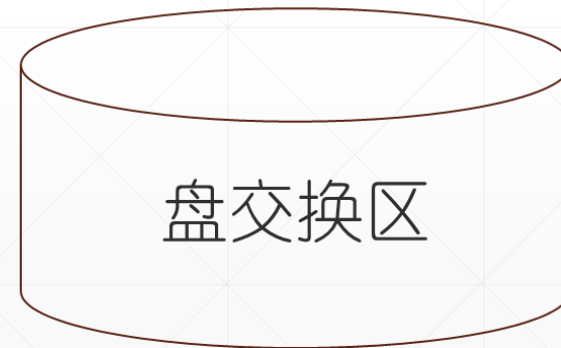
```
class Process
```

```
{
    .....
    int p_flag; // SLOAD 0、1
    int p_time; // 驻留时长
    .....
}
```

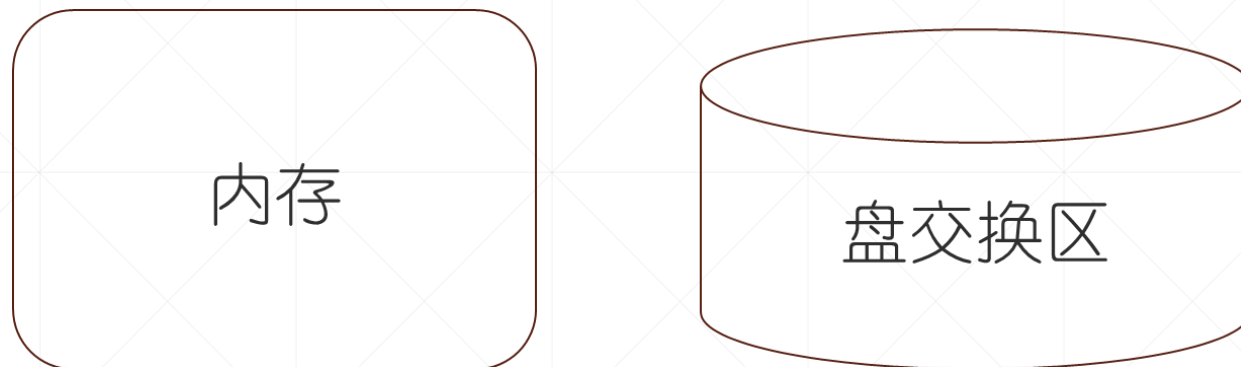


```
class ProcessManager
```

```
{
    .....
    int RunIn; // 盘交换区上有就绪进程，内存无法容纳
    int RunOut; // 内存不挤，盘交换区上没有就绪进程
    .....
}
```



0# 进程



设置0#进程，这个内核服务器负责换入盘交换区上的SRUN

- 盘交换区没有SRUN，0#进程入睡，等待换入任务
RunOut++; Sleep(&RunOut,-100)
- 内存不足，0#进程入睡，等待内存有空
RunIn++; Sleep(&RunIn,-100)



0#进程

- 系统初始化完成后，创建0#进程，执行 Sched()函数。
- 0#进程是核心态进程（内核线程），专门负责换入盘交换区上的进程图像。

```
Sched ( )  
{  
    goto loop;
```

```
Sloop:  
    RunIn++, Sleep(&RunIn);
```

```
loop: // 换入  
    寻找盘交换区p_time最大的就绪进程 PA, second是  
    其驻留时长  
    if (不存在) // 等待换入任务  
        RunOut++; Sleep(&RunOut); goto loop;  
    else  
        为PA分配内存;  
        成功, goto found2
```

不成功，选择换出进程PB、执行对换操作

- 有SWAIT, goto found1;
- 无，考虑换出 SRUN 或 SSLEEP，选p_time最大的（避免抖动，seconds>=3s & PB的 p_time>=2s）找到，goto found1;
 没找到，goto sloop; // 入睡等待内存有空

found1: 换出PB

```
PB->p_flag &= ~Process::SLOAD;  
this->>XSwap(PB, true, 0); // 释放其占用的内存空间  
goto loop; // 尝试换入PA
```

found2: 换入PA

```
PPT 4、5所示换入操作; // 换出操作  
goto loop;
```

```
}
```

1、初始化后的状态

- RunIn是0, RunOut是1。内存足够, 0#进程等待换入任务:
 - **RunOut++ , Sleep(&RunOut);**
- 在盘交换区出现第一个进程前, 0#进程 SSLEEP。其间,
 - 进程切换时, 0#进程执行 Select 选择新运行进程
 - 系统idle时, 0#进程是现运行进程, 等中断
 - 时钟中断, 维护时钟
 - 其它会唤醒进程的中断, 会唤醒睡眠进程。随后, 0#进程选中新就绪进程, select返回后将CPU让给这个进程。回看Swch、Select函数。
- 盘交换区出现第一个进程 PA 后, 系统唤醒0#进程:
 - **WakeUpAll(&Runout);**

盘交换区上的第一个进程是怎么来的?



2、0#进程首次工作

如果正好有SWAIT, 对换成功后, 0#进程 $\text{RunOut}++$; $\text{Sleep}(\&\text{RunOut})$ 。
否则, 内存没空, 盘交换区上有一个SRUN, 0#进程 $\text{RunIn}++$; $\text{Sleep}(\&\text{RunIn})$ 。

```
Sched ( )  
{  
    goto loop;
```

```
Sloop:  
    RunIn++, Sleep(&RunIn);
```

```
loop: // 换入  
    寻找盘交换区p_time最大的就绪进程 PA, second是  
    其驻留时长  
    if (不存在) // 等待换入任务  
        RunOut++; Sleep(&RunOut); goto loop;  
    else  
        为PA分配内存;  
        成功, goto found2
```

不成功, 选择换出进程PB、执行对换操作

- 有SWAIT, goto found1;
- 无, 考虑换出 SRUN 或 SSLEEP, 选p_time最大的
(避免抖动, $\text{seconds} \geq 3\text{s}$ & PB的 $\text{p_time} \geq 2\text{s}$)
找到, goto found1;
没找到, goto sloop; // 入睡等待内存有空

found1: 换出PB

```
PB->p_flag &= ~Process::SLOAD;  
this->>XSwap(PB, true, 0); // 释放其占用的内存空间  
goto loop; // 尝试换入PA
```

found2: 换入PA

```
PPT 5、6 所示换入操作; // 换出操作  
goto loop;
```

```
}
```


3、SWAIT换出自己的进程图像 & 唤醒 0#进程

```
void Process::Sleep(unsigned long chan, int pri)
{
    if ( pri > 0 )
    {
        .....
        if ( procMgr.RunIn != 0 )
        {
            procMgr.RunIn = 0;
            procMgr.WakeUpAll((unsigned long)&procMgr.RunIn);
        }
        .....
    }
}
```

PB唤醒0#进程后入睡放弃CPU。
0#进程上台，换出PB、换入PA（假设PB尺寸大于PA）。
0#进程入睡，PA上台运行。

Sched ()

```
{
    goto loop;
```

Sloop:

RunIn++, Sleep(&RunIn);



loop: // 换入

寻找盘交换区p_time最大的就绪进程 PA,
其驻留时长

if (不存在) // 等待换入任务

RunOut++; Sleep(&RunOut); go

else

为PA分配内存;

成功, **goto found2**



4、中断处理程序唤醒盘交换区上的进程 PD

```
void Process::SetRun()
{
    this->p_wchan = 0;
    this->p_stat = Process::SRUN;

    if ( this->p_pri < procMgr.CurPri )
        procMgr.RunRun++;

    if ( 0 != procMgr.RunOut && (this->p_flag & Process::SLOAD) == 0 )
    {
        procMgr.RunOut = 0;
        procMgr.WakeUpAll((unsigned long)&procMgr.RunOut);
    }
}
```

```
void ProcessManager::WakeUpAll(unsigned long chan)
{
    /* 唤醒系统中所有因chan而进入睡眠的进程 */
    for(int i = 0; i < ProcessManager::NPROC; i++)
        if( this->process[i].IsSleepOn(chan) )
            this->process[i].SetRun();
}
```

现运行进程执行中断处理程序唤醒 PD和0#进程，中断返回时被剥夺，放弃CPU。
0#进程上台（假设内存有足够空间容纳PD），换入PD后入睡放弃CPU。
PD上台运行执行系统调用下半段。。。

5、利用终止进程释放的内存空间

- 每秒结束，Clock 看下 RunIn。如果 RunIn $\neq 0$ ，唤醒0#进程。

```
if ( procMgr.RunIn  $\neq$  0 )  
{  
    procMgr.RunIn = 0;  
    procMgr.WakeUpAll((unsigned long)&procMgr.RunIn);  
}
```

- 被唤醒的0#进程，换入盘交换区上的SRUN进程。如果前1s，有进程终止，释放空间可以用来换入盘交换区上的SRUN进程。



三、习题 1。假设T时刻，内存空间足够容纳所有进程图像

内存中有足够大的空闲区容纳盘交换区上的SRUN进程。

1、T0时刻整数秒，现运行进程 PA正在执行应用程序。 盘交换区上的PB进程定时器waketime到。

- PA响应整数秒时钟中断，唤醒PB，唤醒0#进程，返回用户态前放弃CPU。
- 0#进程上台，换入PB后，入睡放弃CPU Sleep(&RunOut)。
- PB进程上台执行Sleep系统调用下半段，完成后、返回用户态前重算优先数并放弃CPU。
- PA、PB轮流使用CPU，执行应用程序，近似时间片轮转。

[细节，不要求]：其实0#进程换入操作执行期间会启动IO。放弃CPU后PA执行。IO操作完成后再把CPU交还给0#进程，令其修改换入进程PB的状态。



习题 2。假设T时刻，内存空间足够容纳所有进程图像

内存中有足够大的空闲区容纳盘交换区上的SRUN进程。

2、T1时刻整数秒，现运行进程PA正在执行磁盘中断处理程序。盘交换区上的PC进程waketime到。

- T1时刻，PA执行磁盘中断处理程序唤醒等待磁盘IO结束的进程PX。PX上台执行系统调用下半段。T1时刻不会唤醒PC（因为时钟中断的先前态是核心态）。
- 至后续，先前态是用户态的第一次时钟中断（时钟滴答），唤醒PC，唤醒0#进程。时钟中断处理程序执行完毕后，0#进程上台运行，换入PC，放弃CPU。
- PC上台，执行Sleep系统调用下半段，完成后和其它SRUN进程轮流使用CPU。